

Project Overview

Limitly is a distributed rate limiting service built using the MERN stack (MongoDB, Express, React, Node.js). It provides developers with a centralized engine to protect their applications from abuse, brute-force attacks, and traffic spikes by defining custom request limits through API key-based access. The project integrates core DevOps practices — containerization, CI/CD pipelines, version control, and structured deployment workflows — to demonstrate a production-ready development lifecycle.

DevOps Integration

1. Version Control (Git + GitHub)

The entire codebase is managed using Git with GitHub as the remote repository. The project follows a structured branching workflow:

- Feature branches merged into main via pull requests
- .env files excluded via .gitignore; a .env.example is committed for onboarding
- Commit history tracks incremental changes to backend services, Lua scripts, and frontend components

2. Containerization (Docker + Docker Compose)

All services are containerized using Docker to ensure environment consistency across development and production:

- Redis runs in a Docker container (docker run -d --name limitly-redis -p 6379:6379 redis)
- Backend (Node.js/Express) and Frontend (React/Vite) each have their own Dockerfile for isolated builds
- Docker Compose orchestrates the full stack — backend, frontend, MongoDB, and Redis — with a single docker compose up
- Volume mounts ensure MongoDB and Redis data persistence across container restarts

3. CI/CD Pipeline (GitHub Actions)

A GitHub Actions workflow automates the build and test pipeline on every push and pull request:

- Installs dependencies for both backend and frontend (npm install)
- Runs ESLint for code quality checks on every PR
- Builds Docker images to catch containerization issues early
- Runs npm audit to detect known dependency vulnerabilities
- On merge to main: pushes tagged images to GitHub Container Registry (GHCR) and triggers deployment

4. Environment & Configuration Management

Sensitive configuration is managed securely throughout the lifecycle:

- Environment variables (JWT_SECRET, MONGO_URI, REDIS_URL, PORT) stored in .env, never committed
- .env.example documents all required keys for new contributors

- GitHub Actions Secrets used during CI/CD to inject production credentials at runtime
- API keys in MongoDB have a 60-day auto-expiry TTL, enforcing rotation

5. Structured Project Architecture

The codebase follows a clean separation of concerns with dedicated layers for routes, controllers, services, and middleware — enabling independent deployment and scaling of components. Redis is isolated as the rate limiting and analytics store, while MongoDB handles persistent user and key data.

Tech Stack

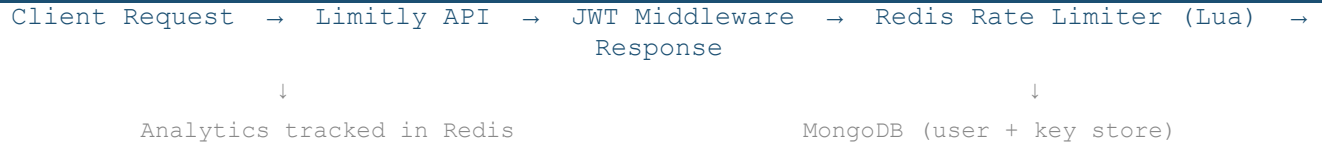
Layer	Technology
Backend	Node.js + Express 5
Database	MongoDB (users, API keys)
Cache / Rate Store	Redis (Lua scripts for atomic ops)
Authentication	JWT (JSON Web Tokens)
Algorithm	Sliding Window Log
Frontend	React + Vite
Containerization	Docker + Docker Compose
CI/CD	GitHub Actions
SCM	Git + GitHub

Key Features

- Sliding Window Log algorithm implemented with Redis + Lua for atomic, race-condition-free rate limiting
- JWT-based user authentication with secure API key generation, listing, and deletion
- Configurable plans: Free (5 req/min) and Pro (100 req/min)
- Real-time analytics dashboard tracking total, allowed, and blocked requests per endpoint and user
- Abuse protection: max 3 API keys per user per day, 60-day auto-expiry, brute-force mitigation
- Middleware-based integration — drop into any Node.js, Express, or Flask application in minutes

System Flow

The end-to-end request lifecycle demonstrates how DevOps and application layers interact:



DevOps Outcomes

- Reproducible environments: any developer can spin up the full stack with docker compose up
- Automated quality gate: linting and security audits run on every pull request via GitHub Actions
- Deployment-ready: Docker images are tagged and pushed to GHCR on every main branch merge
- Secure secret handling: no credentials in source control; GitHub Secrets injected at CI runtime
- Scalable design: Redis and MongoDB run as independent containers, replaceable with managed cloud services